



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

外观模式（Facade Pattern）——嵌入式讲义

1 意图（Intent）

为子系统的一组接口提供一个统一的高层接口，使子系统更易使用。

嵌入式理解

在嵌入式系统中：

- 子系统通常是：
 - 模块组合（Sensor / Display / Comm）
 - 系统服务（Power / RTOS / FileSystem）

👉 Facade 的作用：

把“复杂调用流程”收敛为“简单 API”

2 动机（Motivation）

✗ 问题场景（无 Facade）

例如温控系统：

```
temp = sensor->readValue();
display->showValue(temp);
actuator->setSpeed(speed);
comm->send(data);
```

问题：

- 上层必须知道所有模块
- 必须掌握调用顺序
- 耦合严重
- 难以维护

✓ 引入 Facade

```
Thermostat_RunCycle();
```

内部：

- 读取传感器
- 更新显示
- 控制风扇
- 发送数据

✓ 核心变化

维度	改变
接口数量	多 → 少
复杂度	分散 → 集中
调用者负担	高 → 低

3 适用性 (Applicability)

在嵌入式中，以下场景应使用 Facade：

✓ 1. 子系统复杂

例如：

- Power 管理（时钟 + 外设 + 中断）
- 网络协议栈（TCP/IP）

✓ 2. 调用顺序固定

例如：

初始化 → 采集 → 处理 → 输出

✓ 3. 希望解耦上层

- UI / 业务层 不关心底层细节
- 仅使用简单 API

✓ 4. 分层架构

```
Application
  ↓
Facade ← 推荐放置位置
  ↓
Driver / HAL / Middleware
```

4 结构 (Structure)

标准结构 (嵌入式映射)

```
Client
  ↓
Facade
  ↓
Subsystem A (Sensor)
Subsystem B (Display)
Subsystem C (Comm)
Subsystem D (Actuator)
```

嵌入式代码结构示例

```
// Facade
void Thermostat_RunCycle();

// 子系统
sensor_read();
display_show();
fan_set_speed();
comm_send();
```

5 参与者 (Participants)

1. Facade (外观)

- 提供统一接口
- 封装子系统调用
- 管理调用顺序

👉 示例:

```
void Power_sleep();  
void Device_Start();
```

2. Subsystem Classes (子系统)

- 实现具体功能
- 不知道 Facade 存在

👉 示例:

```
sensor.c  
display.c  
comm.c  
driver.c
```

3. Client (客户端)

- 只调用 Facade
- 不直接操作子系统

6 协作 (Collaborations)

调用流程

```
Client → Facade → Subsystems
```

嵌入式执行链

```
main()  
↓  
Thermostat_RunCycle()  
↓  
sensor_read()  
display_show()  
fan_control()  
comm_send()
```

关键点

- Facade 控制流程
- Client 不参与调度

7 结果 (Consequences)

✓ 优点

1. 降低耦合

- Client 不依赖子系统

2. 简化接口

```
Power_Sleep(); // vs 多个底层调用
```

3. 提高可维护性

- 修改子系统不影响上层

4. 明确分层

- 提供清晰架构边界

✗ 缺点

1. 可能成为“上帝类”

```
System_DoEverything();
```

2. 灵活性降低

- 高级用户可能需要绕过 Facade

8 实现 (Implementation)

✓ 1. 不强制结构形式

在嵌入式中可以是：

- 普通函数

- struct + 函数指针
- 模块接口

👉 关键是“接口层级”，不是语法

✓ 2. Facade 不应过重

建议：

- 按模块拆分 (Power / Device / Net)
- 避免单一巨大接口

✓ 3. 可结合其他模式

常见组合：

模式	作用
Abstract Factory	创建子系统
Singleton	管理全局资源
State	内部状态控制

✓ 4. 推荐结构（嵌入式）

```
Facade
  ↓
Service
  ↓
Driver / HAL
```

9 示例

示例1：Power 管理（经典）

```
void Power_Sleep();
void Power_Wakeup();
```

示例2：LCD 驱动（最常见）

```
LCD_Init();
LCD_DrawPixel();
LCD_Clear();
```

内部：

- SPI
- GPIO
- DMA

示例3：设备控制层

```
Device_Start();  
Device_Process();  
Device_Stop();
```

总结（嵌入式视角）

外观模式不是某种接口形式，而是一种系统边界设计方法。

✓ 核心本质

- 隐藏复杂子系统
- 提供统一入口
- 降低调用复杂度

✓ 在嵌入式中的典型落点

- Power 管理
- LCD / 外设驱动
- 文件系统 / 网络
- 设备控制层

✓ 一句话记忆

Facade = 让系统“更好用”